

StockMaster PWA

Cahier des Charges Technique

Architecture Découplée · Laravel 11 · React.js · PWA

Version :	1.0 – Juin 2026
Type :	Architecture Découplée (API REST + PWA)
Backend :	Laravel 11 · Sanctum · MySQL
Frontend :	React.js · Vite · Axios · PWA
Cibles :	Gestionnaires de stock · Agents d'entrepôt

TABLE DES MATIÈRES

01	Présentation Générale du Projet	Contexte, objectifs, périmètre fonctionnel
02	Architecture Technique	Décision d'architecture, stack technologique, infrastructure
03	Fonctionnalités Clés	Authentification, catalogue, mouvements, traçabilité, PWA
04	Cartographie des Écrans	Spécifications UX/UI des 5 écrans principaux
05	Modèle de Données	Structure BDD, entités, relations, contraintes
06	Spécifications API REST	Endpoints, méthodes HTTP, formats de réponse
07	Règles de Gestion Métier	Lois du code, validations, sécurité, intégrité
08	Sécurité & Authentification	Sanctum, tokens, protection des routes
09	Exigences Non Fonctionnelles	Performance, accessibilité, compatibilité
10	Planning & Livrables	Phases de développement, jalons, critères de succès

01

Présentation Générale du Projet

Contexte · Objectifs · Périmètre · Utilisateurs cibles

1.1 Contexte et Problématique

Les entreprises gérant des stocks physiques (entrepôts, boutiques, PME industrielles) font face à un défi récurrent : la désynchronisation entre les équipes terrain et les gestionnaires back-office. Les outils traditionnels (feuilles Excel, logiciels lourds mono-poste) ne permettent pas une gestion en temps réel depuis n'importe quel appareil.

StockMaster PWA répond à ce besoin en proposant une application web progressive accessible sur PC comme sur smartphone, sans installation depuis les stores mobiles, avec synchronisation instantanée des données.

1.2 Objectifs du Projet

Objectif	Description
Accessibilité universelle	Fonctionner sur PC (bureau) et Smartphone (terrain) via une PWA installable
Traçabilité totale	Journaliser chaque mouvement de stock avec horodatage et identification de l'auteur
Temps réel	Recalcul instantané des quantités après chaque entrée/sortie
Sécurité	Accès restreint par token JWT, sessions persistantes, routes protégées
Performance	Réponse API < 200ms, interface fluide même sur connexion 3G

1.3 Utilisateurs Cibles

Profil	Device Principal	Activités Principales
■ Gestionnaire de Stock	PC Bureau	Administration complète, rapports, configuration catalogue, supervision
■ Agent d'Entrepôt	Smartphone	Saisie rapide de mouvements, consultation de stock, scan terrain

1.4 Périmètre Fonctionnel (V1)

- ✓ Authentification sécurisée par token (connexion / déconnexion / persistance de session)
- ✓ Gestion du catalogue : Catégories (CRUD) et Produits (CRUD + SKU unique)
- ✓ Mouvements de stock : Entrées et Sorties avec calcul automatique des quantités
- ✓ Tableau de bord : Indicateurs clés, alertes stock critique, graphique hebdomadaire
- ✓ Historique des flux : Journal chronologique immuable de tous les événements
- ✓ Interface PWA installable sur Android et iOS

Hors périmètre V1 :

- ✗ Gestion multi-entrepôts (V2)
- ✗ Application mobile native (React Native)
- ✗ Intégration ERP/SAP
- ✗ Gestion des commandes fournisseurs

02

Architecture Technique

Stack technologique · Décisions d'architecture · Infrastructure

2.1 Architecture Découplée (Headless)

Le projet adopte une architecture **découplée** : le backend expose une API REST indépendante que n'importe quel client (web, mobile, tiers) peut consommer. Le frontend React communique exclusivement via HTTP avec cette API, sans aucun couplage de code.

Couche	Technologie	Version	Rôle
Backend API	Laravel + Sanctum	11.x	API REST, logique métier, authentification token, ORM Eloquent
Base de données	MySQL	8.0+	Persistance des données, contraintes d'intégrité référentielle
Frontend PWA	React.js + Vite	18.x + 5.x	Interface utilisateur, gestion d'état, Service Worker PWA
HTTP Client	Axios	1.x	Requêtes API depuis le frontend, intercepteurs d'erreurs
Authentification	Laravel Sanctum	3.x	Token Bearer, protection des routes API, SPA tokens

2.2 Justification des Choix Technologiques

Laravel 11	Framework PHP mature, ORM Eloquent puissant, Sanctum intégré, migrations BDD, artisan CLI
React.js	Écosystème riche, composants réutilisables, performance Virtual DOM, PWA via Vite Plugin
MySQL	SGBD relationnel robuste, transactions ACID, parfaitement supporté par Eloquent
Architecture découplée	Séparation des responsabilités, testabilité indépendante, scalabilité future
PWA	Installable sans Store, fonctionne offline (partiel), push notifications futures, 1 seule codebase

2.3 Flux de Communication

Chaque requête du frontend suit ce cycle : **(1)** L'utilisateur déclenche une action UI → **(2)** Axios envoie une requête HTTP avec le token Bearer en en-tête → **(3)** Laravel valide le token via Sanctum, traite la logique métier → **(4)** Réponse JSON standardisée retournée → **(5)** React met à jour l'état et re-rend l'interface.

03

Fonctionnalités Clés

Spécifications détaillées des 5 modules fonctionnels

F1 — Authentification Sécurisée

Le système d'authentification repose sur **Laravel Sanctum** qui émet des tokens d'accès personnels (Personal Access Tokens). Ces tokens sont stockés côté client dans le **localStorage** et envoyés dans chaque requête via l'en-tête HTTP Authorization: Bearer {token}.

- Connexion : Email + Mot de passe → Token Bearer retourné
- Déconnexion : Révocation du token côté serveur (DELETE /api/logout)
- Persistance : Token stocké localement, session maintenue après refresh
- Protection : Redirection automatique vers /login si token expiré ou absent
- Middleware : auth:sanctum appliqué sur toutes les routes API protégées

F2 — Gestion du Catalogue (Catégories & Produits)

Le catalogue est structuré en deux entités hiérarchiques :

Entité	Champs Clés	Contraintes
Catégorie	id, nom, description, slug, timestamps	Nom unique · Suppression bloquée si produits associés
Produit	id, sku, nom, description, prix_achat, prix_vente, quantité, categorie_id	SKU unique · quantité >= 0 · FK catégorie

Règle critique : La quantité d'un produit n'est jamais modifiable directement. Elle est calculée automatiquement par le système à partir de l'historique des mouvements.

F3 — Mouvements de Stock Intelligents

Chaque mouvement (Entrée ou Sortie) déclenche une transaction SQL atomique garantissant la cohérence des données. Le backend recalcule et met à jour la quantité du produit dans la même transaction.

Type	Effet sur Quantité	Validation Requise	Exemple de Motif
Entrée ■	quantité += valeur saisie	valeur > 0	Réception fournisseur, Retour client
Sortie ■	quantité -= valeur saisie	valeur > 0 ET quantité suffisante	Vente, Casse, Inventaire

F4 — Traçabilité Totale (Journal Immuable)

Chaque mouvement crée un enregistrement permanent dans la table stock_mouvements. Aucun enregistrement ne peut être modifié ou supprimé après création. Ce journal constitue l'audit trail complet du système.

- Qui : user_id (FK vers la table users, relation Eloquent)
- Quand : created_at horodaté automatiquement par Laravel
- Quoi : product_id, type (entrée/sortie), quantité, quantité_avant, quantité_après
- Pourquoi : champ motif (texte libre, obligatoire)

F5 — Mode PWA (Progressive Web App)

L'application frontend est configurée comme une PWA via le plugin **Vite PWA Plugin**. Cela permet l'installation sur l'écran d'accueil Android/iOS sans passer par le Play Store ou l'App Store.

- manifest.webmanifest : nom, icônes, couleurs, display: standalone
- Service Worker : cache des assets statiques pour chargement instantané
- HTTPS requis en production (obligation PWA)
- Meta viewport optimisé pour usage tactile
- Interface responsive : grille CSS adaptée à toutes les tailles d'écran

04

Cartographie des Écrans

Spécifications UX/UI · Comportement PC & Mobile · Composants

E1 — Page de Connexion (Login)

Route	/login
Accès	Public
Description	Écran d'entrée de l'application. Formulaire épuré centré sur fond neutre. Gestion des erreurs inline (mauvais identifiants, champs vides). Redirection automatique vers le Dashboard si token valide déjà présent.
Vue PC	Carte centrée (max-width: 420px) avec ombre portée, logo en en-tête de carte
Vue Mobile	Affichage plein écran, champs larges optimisés pour le pouce, clavier numérique sur champ password

Composants / Éléments UI :

- Input Email + validation
- Input Password (toggle visibilité)
- Bouton de connexion (état loading)
- Message d'erreur dynamique (rouge)
- Logo / branding

E2 — Tableau de Bord (Dashboard)

Route	/dashboard
Accès	Authentifié
Description	Vue d'ensemble après connexion. Doit permettre une lecture immédiate de l'état du stock sans navigation supplémentaire. Données chargées via API au montage du composant.
Vue PC	Grille 3 colonnes pour les KPI cards + tableau d'alertes à droite + graphique pleine largeur en bas
Vue Mobile	Empilage vertical des KPI cards (1 par ligne), tableau d'alertes en accordéon scrollable

Composants / Éléments UI :

- Card KPI : Total produits
- Card KPI : Valeur stock (€)
- Card KPI : Mouvements du jour
- Liste alertes stock critique (< seuil)
- Graphique Entrées vs Sorties (7 jours)

E3 — Gestion du Catalogue

Route	/catalogue
Accès	Authentifié
Description	Écran principal d'administration. Permet la consultation, l'ajout, la modification et la suppression des produits et catégories. Inclut un système de filtres et de recherche en temps réel.
Vue PC	Grand tableau avec colonnes (SKU, Nom, Catégorie, Prix, Quantité, Actions), filtres persistants en en-tête, modal d'ajout/édition
Vue Mobile	Liste de cards verticales empilées avec info condensée, bouton FAB (+) pour ajout, swipe actions pour édition/suppression

Composants / Éléments UI :

- Tableau / Liste de produits
- Barre de recherche (debounce 300ms)
- Filtre par catégorie (dropdown)
- Modal formulaire Produit (ajout/édition)
- Modal formulaire Catégorie
- Badges quantité (vert/orange/rouge selon seuil)

E4 — Module Mouvement de Stock

Route	/mouvements/nouveau
Accès	Authentifié
Description	Écran 'terrain' optimisé pour une utilisation rapide par les agents d'entrepôt. Formulaire minimaliste focalisé sur l'action. Confirmation visuelle immédiate après validation.
Vue PC	Formulaire centré (max-width: 600px), liste déroulante produit avec recherche intégrée
Vue Mobile	Interface XXL avec grands boutons tactiles, stepper quantité (+/-), clavier numérique déclenché automatiquement

Composants / Éléments UI :

- Searchable Select : Produit
- Toggle Entrée/Sortie (visuel coloré)
- Stepper Quantité avec +/- buttons
- Textarea Motif (obligatoire)
- Aperçu stock avant/après
- Bouton Valider (confirmation modale)

E5 — Historique des Flux

Route	/historique
Accès	Authentifié
Description	Journal d'audit complet. Lecture seule. Affichage chronologique inversé (plus récent en premier). Filtres par produit, type de mouvement, utilisateur, période.
Vue PC	Tableau dense avec toutes les colonnes, filtres avancés en sidebar ou en-tête, export CSV (futur)
Vue Mobile	Timeline verticale condensée, chaque ligne en card expandable pour voir le détail complet

Composants / Éléments UI :

- Tableau / Timeline d'événements
- Filtre période (date picker)
- Filtre type (Entrée/Sortie/Tous)
- Filtre produit (searchable)
- Badge coloré type mouvement
- Pagination (20 items/page)

05

Modèle de Données

Structure BDD MySQL · Entités · Relations · Contraintes

◆ Table : users — Utilisateurs de l'application (gestionnaires et agents)

Colonne	Type	Contraintes / Notes
id	BIGINT UNSIGNED	PK, Auto-increment
name	VARCHAR(255)	NOT NULL
email	VARCHAR(255)	NOT NULL, UNIQUE
password	VARCHAR(255)	NOT NULL, Bcrypt hashé
role	ENUM('admin','agent')	NOT NULL, DEFAULT 'agent'
email_verified_at	TIMESTAMP	NULLABLE
created_at / updated_at	TIMESTAMP	Auto-managed by Laravel

◆ Table : categories — Classification des produits

Colonne	Type	Contraintes / Notes
id	BIGINT UNSIGNED	PK, Auto-increment
nom	VARCHAR(150)	NOT NULL, UNIQUE
description	TEXT	NULLABLE
slug	VARCHAR(160)	NOT NULL, UNIQUE
created_at / updated_at	TIMESTAMP	Auto-managed

◆ Table : products — Catalogue des produits stockés

Colonne	Type	Contraintes / Notes
id	BIGINT UNSIGNED	PK, Auto-increment
categorie_id	BIGINT UNSIGNED	FK → categories.id, RESTRICT ON DELETE
sku	VARCHAR(100)	NOT NULL, UNIQUE — Identifiant unique produit
nom	VARCHAR(255)	NOT NULL
description	TEXT	NULLABLE
prix_achat	DECIMAL(10,2)	NOT NULL, DEFAULT 0.00
prix_vente	DECIMAL(10,2)	NOT NULL, DEFAULT 0.00
quantite	INT UNSIGNED	NOT NULL, DEFAULT 0 — Calculé automatiquement
seuil_alerte	INT UNSIGNED	NOT NULL, DEFAULT 5
created_at / updated_at	TIMESTAMP	Auto-managed

◆ Table : stock_movements — Journal immuable de tous les mouvements de stock

Colonne	Type	Contraintes / Notes
id	BIGINT UNSIGNED	PK, Auto-increment
product_id	BIGINT UNSIGNED	FK → products.id, RESTRICT ON DELETE
user_id	BIGINT UNSIGNED	FK → users.id, RESTRICT ON DELETE
type	ENUM('entree','sortie')	NOT NULL
quantite	INT UNSIGNED	NOT NULL, CHECK (quantite > 0)
quantite_avant	INT UNSIGNED	NOT NULL — Snapshot avant mouvement
quantite_apres	INT UNSIGNED	NOT NULL — Snapshot après mouvement
motif	VARCHAR(500)	NOT NULL
created_at	TIMESTAMP	NOT NULL — Immuable, pas d'updated_at

Relations (Eloquent Relationships)

Relation	Type Eloquent	Description
User → StockMovement	hasMany	Un utilisateur peut avoir plusieurs mouvements
Category → Product	hasMany	Une catégorie contient plusieurs produits
Product → Category	belongsTo	Un produit appartient à une catégorie
Product → StockMovement	hasMany	Un produit a plusieurs mouvements dans son historique
StockMovement → User	belongsTo	Chaque mouvement est lié à son auteur
StockMovement → Product	belongsTo	Chaque mouvement concerne un produit

06

Spécifications API REST

Endpoints · Méthodes HTTP · Paramètres · Formats de réponse

6.1 Conventions Générales

Base URL	https://api.stockmaster.app/api
Format	JSON exclusivement (Content-Type: application/json)
Auth	Authorization: Bearer {token} sur toutes routes protégées
Pagination	?page=1&per_page=20 sur les listes (Laravel Pagination)
Erreurs	HTTP Status codes standard + corps JSON {message, errors}
Versioning	Préfixe /api/v1/ en production

6.2 Endpoints par Module

Authentification

Méthode	Endpoint	Auth	Description
POST	/api/login	Non	Connexion, retourne {token, user}
POST	/api/logout	Oui	Révocation du token courant
GET	/api/user	Oui	Profil de l'utilisateur connecté

Catégories

Méthode	Endpoint	Auth	Description
GET	/api/categories	Oui	Liste toutes les catégories
POST	/api/categories	Oui	Créer une catégorie
GET	/api/categories/{id}	Oui	Détail d'une catégorie
PUT	/api/categories/{id}	Oui	Modifier une catégorie
DELETE	/api/categories/{id}	Oui	Supprimer (bloqué si produits actifs)

Produits

Méthode	Endpoint	Auth	Description
GET	/api/products	Oui	Liste paginée (filtre: ?category_id=, ?search=)
POST	/api/products	Oui	Créer un produit (quantité = 0 par défaut)
GET	/api/products/{id}	Oui	Détail + historique récent
PUT	/api/products/{id}	Oui	Modifier infos (pas la quantité directement)
DELETE	/api/products/{id}	Oui	Supprimer (bloqué si mouvements existants)

Mouvements de Stock

Méthode	Endpoint	Auth	Description
GET	/api/movements	Oui	Journal complet paginé (filtre: type, product_id, user_id, date)
POST	/api/movements	Oui	Créer un mouvement → recalcul quantité en transaction
GET	/api/movements/{id}	Oui	Détail d'un mouvement (lecture seule)

Dashboard

Méthode	Endpoint	Auth	Description
GET	/api/dashboard/stats	Oui	Total produits, valeur stock, mouvements du jour
GET	/api/dashboard/alerts	Oui	Produits sous le seuil d'alerte
GET	/api/dashboard/weekly	Oui	Entrées vs Sorties des 7 derniers jours

07

Règles de Gestion Métier

Lois du code · Validations · Contraintes d'intégrité

RG-01 Immutabilité de la Quantité

CRITIQUE

La quantité d'un produit ne peut jamais être modifiée directement via une requête PUT/PATCH. Elle est calculée et mise à jour UNIQUEMENT lors de la création d'un mouvement de stock. Toute tentative directe retourne HTTP 422.

RG-02 Blocage Sortie Insuffisante

CRITIQUE

Avant de valider une sortie, le backend vérifie que `product.quantite >= mouvement.quantite`. Si insuffisant, retourne HTTP 422 avec message 'Stock insuffisant (disponible: X)'. Cette validation se fait en transaction pour éviter les race conditions.

RG-03 Transaction Atomique

HAUTE

La création d'un mouvement et la mise à jour de la quantité produit se font dans une transaction SQL unique (DB::transaction). Si l'une échoue, tout est rollback.

RG-04 SKU Unique Global

HAUTE

Le SKU (Stock Keeping Unit) doit être unique dans toute la table products. Validation Laravel : `Rule::unique('products', 'sku')->ignore($id)` lors des mises à jour.

RG-05 Catégorie Non-Supprimable

HAUTE

Une catégorie ne peut être supprimée si elle contient au moins un produit actif. Laravel protège cette contrainte via FOREIGN KEY RESTRICT. Retourne HTTP 409.

RG-06 Quantité Positive Obligatoire

NORMALE

La quantité d'un mouvement doit être un entier strictement positif (≥ 1). Validation Laravel : `'quantite' => 'required|integer|min:1'`.

RG-07 Motif Obligatoire

NORMALE

Le champ motif est obligatoire pour tout mouvement. Minimum 3 caractères, maximum 500. Permet l'audit et la traçabilité.

RG-08 Redirection Sécurisée

HAUTE

Si le token est absent, expiré ou invalide, Axios intercepte la réponse HTTP 401 et redirige automatiquement vers /login. L'intercepteur Axios est configuré globalement.

08

Sécurité & Authentification

Laravel Sanctum · Protection des routes · Bonnes pratiques

8.1 Mécanisme d'Authentification

Laravel Sanctum est utilisé en mode **Personal Access Tokens** (adapté aux SPA et API). Contrairement aux sessions basées sur les cookies, ce mode émet un token opaque que le client stocke et envoie dans chaque requête.

1. POST /api/login	Le serveur vérifie les credentials (bcrypt::check)
2. Création token	Sanctum génère un token : <code>\$user->createToken('main')->plainTextToken</code>
3. Stockage client	React stocke le token dans localStorage (clé : 'ACCESS_TOKEN')
4. Requêtes suivantes	Axios envoie : <code>Authorization: Bearer {token}</code> sur chaque requête
5. Validation serveur	Middleware auth:sanctum valide le token à chaque requête API
6. Logout	<code>DELETE /api/logout → \$request->user()->currentAccessToken()->delete()</code>

8.2 Protection des Routes Laravel

Dans `routes/api.php`, toutes les routes sensibles sont encapsulées dans un groupe middleware :

```
Route::middleware('auth:sanctum')->group(function () { Route::apiResource('products', ProductController::class); Route::apiResource('categories', CategoryController::class); Route::apiResource('movements', StockMovementController::class); Route::get('dashboard/stats', [DashboardController::class, 'stats']); });
```

8.3 Sécurité Frontend (Axios Interceptors)

Un intercepteur Axios global capture automatiquement les réponses HTTP 401 et nettoie la session locale avant redirection :

```
axiosClient.interceptors.response.use( response => response, error => { if (error.response?.status === 401) { localStorage.removeItem('ACCESS_TOKEN'); window.location.href = '/login'; } return Promise.reject(error); } );
```

8.4 Autres Mesures de Sécurité

- Mots de passe hashés avec bcrypt (coût 12) — jamais stockés en clair
- CORS configuré dans Laravel pour n'autoriser que l'origine du frontend
- Rate limiting sur les routes d'authentification (60 req/min par IP)
- Validation stricte de toutes les entrées utilisateur (Laravel Form Requests)
- Variables d'environnement sensibles dans `.env` (jamais commitées en Git)
- HTTPS obligatoire en production (exigence PWA et sécurité des tokens)

09

Exigences Non Fonctionnelles

Performance · Compatibilité · Accessibilité · Maintenabilité

Catégorie	Exigence	Mesure / Critère
Performance API	Temps de réponse < 200ms (P95)	Mesure avec Laravel Telescope en dev
Performance UI	First Contentful Paint < 1.5s	Vite bundle optimisé, lazy loading routes
Compatibilité Desktop	Chrome 90+, Firefox 88+, Edge 90+, Safari 14+	Tests manuels cross-browser
Compatibilité Mobile	Android 8+ (Chrome), iOS 14+ (Safari)	PWA installable sur les 2 plateformes
Responsive	Breakpoints : 320px, 768px, 1024px, 1440px	CSS Grid + Flexbox, Tailwind CSS
Disponibilité	99.5% uptime mensuel (hors maintenance)	Monitoring serveur (uptime robot)
Sécurité	Aucune donnée sensible en logs	Review de code, Laravel logs filtrés
Maintenabilité	Couverture de tests > 70% (backend)	PHPUnit + Feature tests Laravel
Accessibilité	ARIA labels sur éléments interactifs	Audit Lighthouse > 80/100

10

Planning & Livrables

Phases de développement · Jalons · Critères de succès

Phase 1		Semaine 1	Initialisation Laravel + React, configuration Sanctum, structure BDD (migrations), CORS, Axios config, structure des dossiers
Phase 2		Semaine 1-2	Login/Logout backend + frontend, persistance token, routes protégées, redirection automatique, tests auth
Phase 3		Semaine 2-3	CRUD Catégories + Produits (API + UI), validation SKU, formulaires modals, tableau/liste responsive
Phase 4		Semaine 3-4	Module mouvements (entrée/sortie), transaction atomique, blocage insuffisance, formulaire mobile-first
Phase 5		Semaine 4	Dashboard KPIs + alertes + graphique, historique complet avec filtres, pagination
Phase 6		Semaine 5	Configuration PWA (manifest + SW), tests cross-browser/mobile, optimisations performance, déploiement

Critères de Succès (Définition of Done)

- ✓ Tous les endpoints API retournent les codes HTTP corrects et les données attendues
- ✓ L'application est installable comme PWA sur Android (Chrome) et iOS (Safari)
- ✓ Aucune sortie de stock n'est possible si la quantité est insuffisante
- ✓ Chaque mouvement génère un enregistrement immuable dans stock_movements
- ✓ L'interface est utilisable sur smartphone (écran 375px) sans scroll horizontal
- ✓ La déconnexion révoque le token côté serveur et côté client
- ✓ Lighthouse PWA score > 90/100 en production